

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №5
«ПРОЦЕДУРЫ, ФУНКЦИИ, ТРИГГЕРЫ В POSTGRESQL»
по дисциплине «Проектирование и реализация баз данных»

Обучающийся Сигаева Ксения Леонидовна
Факультет прикладной информатики
Группа К3240
Направление подготовки 09.03.03 Прикладная информатика
Образовательная программа Мобильные и сетевые технологии 2024
Преподаватель Говорова Марина Михайловна

Санкт-Петербург
2025/2026

Цель работы

Овладеть практическими навыками создания и использования процедур, функций и триггеров в базе данных PostgreSQL.

Практическое задание

1. Создать 3 процедуры для индивидуальной базы данных согласно варианту.
2. Создать 7 триггеров для индивидуальной базы данных.
3. Работу выполнить в консоли SQL Shell (psql), привести скрипты кода разработанных объектов и подтверждающие скриншоты работы и результатов.

Выполнение

Создание хранимых процедур

Процедура 1. Снижение цен на книги, остаток которых на базе превышает 1000 экземпляров (процент скидки передается параметром).

До:

```
SELECT e.editions_id, b.title, e.price, c.copies_in_remain
FROM editions e
JOIN books b ON e.book_id = b.book_id
JOIN circulations c ON e.editions_id = c.editions_id
WHERE c.copies_in_remain > 1000
ORDER BY e.editions_id;
```

editions_id	title	price	copies_in_remain
1	Python: полное руководство	1381	1200
3	Java: эффективное программирование	1093	2500
6	Функциональное программирование на Scala	863	3200
8	Умные контракты и блокчейн на Solidity	1633	1800
11	SQL для аналитики данных	1323	1500
13	Введение в алгоритмы	1024	2800
16	Грокаем алгоритмы	1495	2200
18	Современная веб-разработка	1058	1300
20	Android: разработка приложений на Kotlin	2243	1600
21	Современный JavaScript для профессионалов	920	1700
23	Vue.js в действии	1783	2600
25	Бэкенд-разработка на Go	2760	1100
26	Разработка API с FastAPI	1438	2400
28	Fullstack-разработка на JavaScript	1116	1400
30	Искусственный интеллект и нейронные сети	2358	1650
31	Глубокое обучение на TensorFlow	1357	1550
33	Анализ данных в Excel и Power BI	909	2700
35	Основы DevOps и CI/CD	2300	1050
36	Docker и Kubernetes для начинающих	1587	23-- Далее
50			
38	Облачные технологии: AWS для разработчиков	1081	1450-- Дал
40	Безопасность веб-приложений	2162	1580-- Дал
41	Кибербезопасность: защита данных	1403	1680-- Дал
43	Linux для администраторов	1001	2750-- Дал
45	Компьютерные сети	2473	1080-- Дал
46	Тестирование ПО: от теории к практике	1484	2300-- Дал
48	Микросервисная архитектура	1047	1350-- Дал
(26 строк)			

Процедура:

```
CREATE OR REPLACE PROCEDURE decrease_price (  
    IN p_percent numeric DEFAULT 10  
)  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    UPDATE editions  
    SET price = price - price * p_percent / 100  
    WHERE editions_id IN (  
        SELECT e.editions_id  
        FROM editions e  
        JOIN circulations c ON e.editions_id = c.editions_id  
        WHERE c.copies_in_remain > 1000  
    );  
END;  
$$;
```

Вызов процедуры:

```
CALL decrease_price(10);
```

После:

editions_id	title	price	copies_in_remain
1	Python: полное руководство	1243	1200
3	Java: эффективное программирование	984	2500
6	Функциональное программирование на Scala	777	3200
8	Умные контракты и блокчейн на Solidity	1470	1800
11	SQL для аналитики данных	1191	1500
13	Введение в алгоритмы	922	2800
16	Грокаем алгоритмы	1346	2200
18	Современная веб-разработка	952	1300
20	Android: разработка приложений на Kotlin	2019	1600
21	Современный JavaScript для профессионалов	828	1700
23	Vue.js в действии	1605	2600
25	Бэкенд-разработка на Go	2484	1100
26	Разработка API с FastAPI	1294	2400
28	Fullstack-разработка на JavaScript	1004	1400
30	Искусственный интеллект и нейронные сети	2122	1650
31	Глубокое обучение на TensorFlow	1221	1550
33	Анализ данных в Excel и Power BI	818	2700
35	Основы DevOps и CI/CD	2070	1050
36	Docker и Kubernetes для начинающих	1428	2350
38	Облачные технологии: AWS для разработчиков	973	1450
40	Безопасность веб-приложений	1946	1580
41	Кибербезопасность: защита данных	1263	1680
43	Linux для администраторов	901	2750
45	Компьютерные сети	2226	1080
46	Тестирование ПО: от теории к практике	1336	2300
48	Микросервисная архитектура	942	1350

(26 строк)

Процедура 2. Удаление заказов со статусом «отменен»

Перед удалением договора очищаются зависимые строки в таблицах `accounts` и `contract_content`.

До:

```
SELECT contract_id, customer_id, manager_id, status
FROM contracts
WHERE status = 'отменен';
```

contract_id	customer_id	manager_id	status
103	1	1	отменен

(1 строка)

Процедура:

```
CREATE OR REPLACE PROCEDURE delete_cancelled_contracts()
LANGUAGE plpgsql
AS $$
BEGIN
    DELETE FROM accounts
    WHERE contract_id IN (
        SELECT contract_id
        FROM contracts
        WHERE status = 'отменен'
    );

    DELETE FROM contract_content
    WHERE contract_id IN (
        SELECT contract_id
        FROM contracts
        WHERE status = 'отменен'
    );

    DELETE FROM contracts
    WHERE status = 'отменен';
END;
$$;
```

Вызов процедуры:

```
CALL delete_cancelled_contracts();
```

После:

contract_id	customer_id	manager_id	status
-------------	-------------	------------	--------

(0 строк)

Процедура 3. Ввод нового заказа

Процедура добавляет новый договор с текущей датой создания и заданной датой выполнения.

До:

```
SELECT MAX(contract_id)
FROM contracts;
```

```
max
-----
102
(1 строка)
```

Процедура:

```
CREATE OR REPLACE PROCEDURE add_new_contract (
    IN p_customer_id bigint,
    IN p_manager_id bigint,
    IN p_date_fulfillment date,
    IN p_status varchar DEFAULT 'подписан'
)
LANGUAGE plpgsql
AS $$
BEGIN
    INSERT INTO contracts (
        customer_id,
        manager_id,
        date_creation,
        status,
        date_fulfillment,
        date_completion
    )
    VALUES (
        p_customer_id,
        p_manager_id,
        CURRENT_DATE,
        p_status,
        p_date_fulfillment,
        NULL
    );
END;
$$;
```

Вызов процедуры:

```
CALL add_new_contract(1, 1, CURRENT_DATE + 14, 'подписан');
```

После:

```
SELECT contract_id, customer_id, manager_id, date_creation, status,
date_fulfillment
FROM contracts
WHERE contract_id = (SELECT MAX(contract_id) FROM contracts);
```

contract_id	customer_id	manager_id	date_creation	status	date_fulfillment
104	1	1	2026-05-24	подписан	2026-06-07

(1 строка)

Создание триггеров

Для реализации триггеров сначала создается таблица аудита.

```
CREATE TABLE IF NOT EXISTS contracts_audit (
  audit_id bigserial PRIMARY KEY,
  operation_name varchar(10) NOT NULL,
  operation_time timestamp NOT NULL DEFAULT now(),
  contract_id bigint,
  old_status varchar(13),
  new_status varchar(13),
  old_date_fulfillment date,
  new_date_fulfillment date,
  old_date_completion date,
  new_date_completion date
);

CREATE TABLE IF NOT EXISTS editions_price_audit (
  audit_id bigserial PRIMARY KEY,
  operation_time timestamp NOT NULL DEFAULT now(),
  editions_id bigint NOT NULL,
  old_price numeric(10,0),
  new_price numeric(10,0)
);
```

Триггер 1. Аудит добавления, изменения и удаления заказов

Функция:

```
CREATE OR REPLACE FUNCTION fn_contracts_audit()
RETURNS trigger
LANGUAGE plpgsql
AS $$
BEGIN
  IF TG_OP = 'INSERT' THEN
    INSERT INTO contracts_audit (
      operation_name, contract_id, new_status,
      new_date_fulfillment, new_date_completion
    )
    VALUES (
      TG_OP, NEW.contract_id, NEW.status,
      NEW.date_fulfillment, NEW.date_completion
    );
    RETURN NEW;
  ELSIF TG_OP = 'UPDATE' THEN
    INSERT INTO contracts_audit (
      operation_name, contract_id, old_status, new_status,
```

```

        old_date_fulfillment, new_date_fulfillment,
        old_date_completion, new_date_completion
    )
VALUES (
    TG_OP, NEW.contract_id, OLD.status, NEW.status,
    OLD.date_fulfillment, NEW.date_fulfillment,
    OLD.date_completion, NEW.date_completion
);
RETURN NEW;
ELSIF TG_OP = 'DELETE' THEN
    INSERT INTO contracts_audit (
        operation_name, contract_id, old_status,
        old_date_fulfillment, old_date_completion
    )
VALUES (
        TG_OP, OLD.contract_id, OLD.status,
        OLD.date_fulfillment, OLD.date_completion
    );
RETURN OLD;
END IF;
END;
$$;

```

Триггер:

```

CREATE TRIGGER trg_contracts_audit
AFTER INSERT OR UPDATE OR DELETE ON contracts
FOR EACH ROW
EXECUTE PROCEDURE fn_contracts_audit();

```

Удаление триггера:

```

DROP TRIGGER IF EXISTS trg_contracts_audit ON contracts;

```

После:

```

UPDATE contracts
SET status = status
WHERE contract_id = (SELECT MAX(contract_id) FROM contracts);

SELECT *
FROM contracts_audit
ORDER BY audit_id DESC;

```

audit_id	operation_name	operation_time	contract_id	old_status	new_status	old_date_fulfillment	new_date_fulfillment
1	UPDATE	2026-05-25 12:05:27.88513	104	подписан	подписан	2026-06-07	2026-06-07

(1 строка)

Триггер 2. Аудит изменения цены издания

Функция:

```

CREATE OR REPLACE FUNCTION fn_editions_price_audit()
RETURNS trigger
LANGUAGE plpgsql
AS $$

```

```

BEGIN
    IF OLD.price <> NEW.price THEN
        INSERT INTO editions_price_audit (editions_id, old_price, new_price)
        VALUES (OLD.editions_id, OLD.price, NEW.price);
    END IF;

    RETURN NEW;
END;
$$;

```

Триггер:

```

CREATE TRIGGER trg_editions_price_audit
AFTER UPDATE OF price ON editions
FOR EACH ROW
EXECUTE PROCEDURE fn_editions_price_audit();

```

Удаление триггера:

```

DROP TRIGGER IF EXISTS trg_editions_price_audit ON editions;

```

После:

```

UPDATE editions
SET price = price + 100
WHERE editions_id = 3;

SELECT *
FROM editions_price_audit
ORDER BY audit_id DESC;

```

audit_id	operation_time	editions_id	old_price	new_price
1	2026-05-25 12:22:56.439719	3	984	1084

(1 строка)

Триггер 3. Увеличение количества договоров менеджера при добавлении заказа

До:

manager_id	number_of_contracts
1	3

(1 строка)

Функция:

```

CREATE OR REPLACE FUNCTION fn_manager_contracts_after_insert()
RETURNS trigger
LANGUAGE plpgsql
AS $$
BEGIN
    UPDATE managers

```



```
SET number_of_contracts = number_of_contracts + 1
WHERE manager_id = NEW.manager_id;
RETURN NEW;
END;
$$;
```

Триггер:

```
CREATE TRIGGER trg_manager_contracts_after_insert
AFTER INSERT ON contracts
FOR EACH ROW
EXECUTE PROCEDURE fn_manager_contracts_after_insert();
```

Удаление триггера:

```
DROP TRIGGER IF EXISTS trg_manager_contracts_after_insert ON contracts;
```

После:

```
INSERT INTO contracts (customer_id, manager_id, date_creation, status,
date_fulfillment, date_completion)
VALUES (1, 1, CURRENT_DATE, 'подписан', CURRENT_DATE + 10, NULL);

SELECT manager_id, number_of_contracts
FROM managers
WHERE manager_id = 1;
```

manager_id	number_of_contracts
1	4

(1 строка)

Триггер 4. Уменьшение количества договоров менеджера при удалении заказа

До:

manager_id	number_of_contracts
1	4

(1 строка)

Функция:

```
CREATE OR REPLACE FUNCTION fn_manager_contracts_after_delete()
RETURNS trigger
LANGUAGE plpgsql
AS $$
BEGIN
```

```

UPDATE managers
SET number_of_contracts = number_of_contracts - 1
WHERE manager_id = OLD.manager_id
AND number_of_contracts > 0;

RETURN OLD;
END;
$$;

```

Триггер:

```

CREATE TRIGGER trg_manager_contracts_after_delete
AFTER DELETE ON contracts
FOR EACH ROW
EXECUTE PROCEDURE fn_manager_contracts_after_delete();

```

Удаление триггера:

```

DROP TRIGGER IF EXISTS trg_manager_contracts_after_delete ON contracts;

```

После:

```

DELETE FROM contracts
WHERE contract_id = (SELECT MAX(contract_id) FROM contracts);

SELECT manager_id, number_of_contracts
FROM managers
WHERE manager_id = 1;

```

manager_id	number_of_contracts
1	3

(1 строка)

Триггер 5. Создание нового тиража при снижении остатка до нуля

До:

circulation_id	number_of_copies	requirement_id	editions_id	copies_in_remain
1	5000	1	1	1200

(1 строка)

Функция:

```

CREATE OR REPLACE FUNCTION fn_create_new_circulation_when_empty()
RETURNS trigger
LANGUAGE plpgsql
AS $$
BEGIN
    IF NEW.copies_in_remain <= 0 AND OLD.copies_in_remain > 0 THEN
        INSERT INTO circulations (

```

```

        number_of_copies,
        requirement_id,
        editions_id,
        copies_in_remain
    )
    OVERRIDING SYSTEM VALUE
    VALUES (
        OLD.number_of_copies,
        OLD.requirement_id,
        OLD.editions_id,
        OLD.number_of_copies
    );
END IF;

RETURN NEW;
END;
$$;
$$;

```

Триггер:

```

CREATE TRIGGER trg_create_new_circulation_when_empty
AFTER UPDATE OF copies_in_remain ON circulations
FOR EACH ROW
EXECUTE PROCEDURE fn_create_new_circulation_when_empty();

```

Удаление триггера:

```

DROP TRIGGER IF EXISTS trg_create_new_circulation_when_empty ON circulations;

```

После:

```

UPDATE circulations
SET copies_in_remain = 0
WHERE circulation_id = 1;

SELECT circulation_id, number_of_copies, requirement_id, editions_id,
copies_in_remain
FROM circulations
WHERE editions_id = (SELECT editions_id FROM circulations WHERE circulation_id =
1)
ORDER BY circulation_id;

```

circulation_id	number_of_copies	requirement_id	editions_id	copies_in_remain
1	5000	1	1	0
164	5000	1	1	5000

(2 строки)

Триггер 6. Уменьшение остатка тиража после добавления строки заказа

До:

circulation_id	copies_in_remain
2	500

(1 строка)

Функция:

```
CREATE OR REPLACE FUNCTION fn_decrease_circulation_remain()
RETURNS trigger
LANGUAGE plpgsql
AS $$
BEGIN
    UPDATE circulations
    SET copies_in_remain = copies_in_remain - NEW.number_of_books
    WHERE circulation_id = NEW.circulation_id;

    RETURN NEW;
END;
$$;
```

Триггер:

```
CREATE TRIGGER trg_decrease_circulation_remain
AFTER INSERT ON contract_content
FOR EACH ROW
EXECUTE PROCEDURE fn_decrease_circulation_remain();
```

Удаление триггера:

```
DROP TRIGGER IF EXISTS trg_decrease_circulation_remain ON contract_content;
```

После:

```
INSERT INTO contract_content (contract_id, circulation_id, number_of_books,
period)
VALUES ((SELECT MAX(contract_id) FROM contracts), 2, 10, 30);

SELECT circulation_id, copies_in_remain
FROM circulations
WHERE circulation_id = 2;
```

circulation_id	copies_in_remain
2	490

(1 строка)

Триггер 7. Восстановление остатка тиража при удалении строки заказа

До:

circulation_id	copies_in_remain
2	490

(1 строка)

Функция:

```
CREATE OR REPLACE FUNCTION fn_restore_circulation_remain()
RETURNS trigger
LANGUAGE plpgsql
AS $$
BEGIN
    UPDATE circulations
    SET copies_in_remain = copies_in_remain + OLD.number_of_books
    WHERE circulation_id = OLD.circulation_id;

    RETURN OLD;
END;
$$;
```

Триггер:

```
CREATE TRIGGER trg_restore_circulation_remain
AFTER DELETE ON contract_content
FOR EACH ROW
EXECUTE PROCEDURE fn_restore_circulation_remain();
```

Удаление триггера:

```
DROP TRIGGER IF EXISTS trg_restore_circulation_remain ON contract_content;
```

После:

```
DELETE FROM contract_content
WHERE contract_content_id = (SELECT MAX(contract_content_id) FROM
contract_content);

SELECT circulation_id, copies_in_remain
FROM circulations
WHERE circulation_id = 2;
```

circulation_id	copies_in_remain
2	500
(1 строка)	

Выводы

В ходе лабораторной работы были созданы хранимые процедуры для изменения цен, удаления отмененных заказов и ввода нового заказа. Также были разработаны триггеры, обеспечивающие аудит изменений, проверку корректности дат, автоматическое обновление количества договоров у менеджеров и контроль остатков тиражей при добавлении строк заказа.

Использование процедур позволяет вынести повторяющиеся операции на уровень базы данных, а триггеры помогают автоматически поддерживать целостность данных и фиксировать важные изменения без дополнительного ручного контроля.